QUIKE SORT , AN APPLICATION OF STACKS

6.5 QUICKSORT, AN APPLICATION OF STACKS

Let A be a list of n data items. "Sorting A" refers to the operation of rearranging the elements of A so that they are in some logical order, such as numerically ordered when A contains numerical data, or alphabetically ordered when A contains character data. The subject of sorting, including various sorting algorithms, is treated mainly in Chap. 9. This section gives only one sorting algorithm, called *quicksort*, in order to illustrate an application of stacks.

Quicksort is an algorithm of the divide-and-conquer type. That is, the problem of sorting a set is reduced to the problem of sorting two smaller sets. We illustrate this "reduction step" by means of a specific example.

Suppose A is the following list of 12 numbers:

(44), 33, 11, 55, 77, 90, 40, 60, 99, 22, 88, (66)

The reduction step of the quicksort algorithm finds the final position of one of the numbers; in this illustration, we use the first number, 44. This is accomplished as follows. Beginning with the last number, 66, scan the list from right to left, comparing each number with 44 and stopping at the first number less than 44. The number is 22. Interchange 44 and 22 to obtain the list

(22), 33, 11, 55, 77, 90, 40, 60, 99, (44), 88, 66

(Observe that the numbers 88 and 66 to the right of 44 are each greater than 44.) Beginning with 22, next scan the list in the opposite direction, from left to right, comparing each number with 44 and stopping at the first number greater than 44. The number is 55. Interchange 44 and 55 to obtain the list

22, 33, 11, (44), 77, 90, 40, 60, 99, (55), 88, 66

(Observe that the numbers 22, 33 and 11 to the left of 44 are each less than 44.) Beginning this time with 55, now scan the list in the original direction, from right to left, until meeting the first number less than 44. It is 40. Interchange 44 and 40 to obtain the list

22, 33, 11, (40), 77, 90, (44), 60, 99, 55, 88, 66

(Again, the numbers to the right of 44 are each greater than 44.) Beginning with 40, scan the list from left to right. The first number greater than 44 is 77. Interchange 44 and 77 to obtain the list

22, 33, 11, 40, (44), 90, (77), 60, 99, 55, 88, 66

(Again, the numbers to the left of 44 are each less than 44.) Beginning with 77, scan the list from right to left seeking a number less than 44. We do not meet such a number before meeting 44. This means all numbers have been scanned and compared with 44. Furthermore, all numbers less than 44 now form the sublist of numbers to the left of 44, and all numbers greater than 44 now form the sublist of numbers to the right of 44, as shown below:

22, 33, 11, 40, (44), 90, 77, 60, 99, 55, 88, 66.

First sublist — Second sublist

Thus 44 is correctly placed in its final position, and the task of sorting the original list A has now been reduced to the task of sorting each of the above sublists.

The above reduction step is repeated with each sublist containing 2 or more elements. Since we can process only one sublist at a time, we must be able to keep track of some sublists for future processing. This is accomplished by using two stacks, called LOWER and UPPER, to temporarily "hold" such

MERGE SORT

MERGE SORT

Merging is the process of combining two sorted lists into single sorted list. To perform the merge sort both the sorted lists are compared. The smaller of both the elements is stored in the third array. The sorting completes when all the elements from both the lists are placed in the third list.

Figure 11.5 explains the merge sort.

MERGE SORT

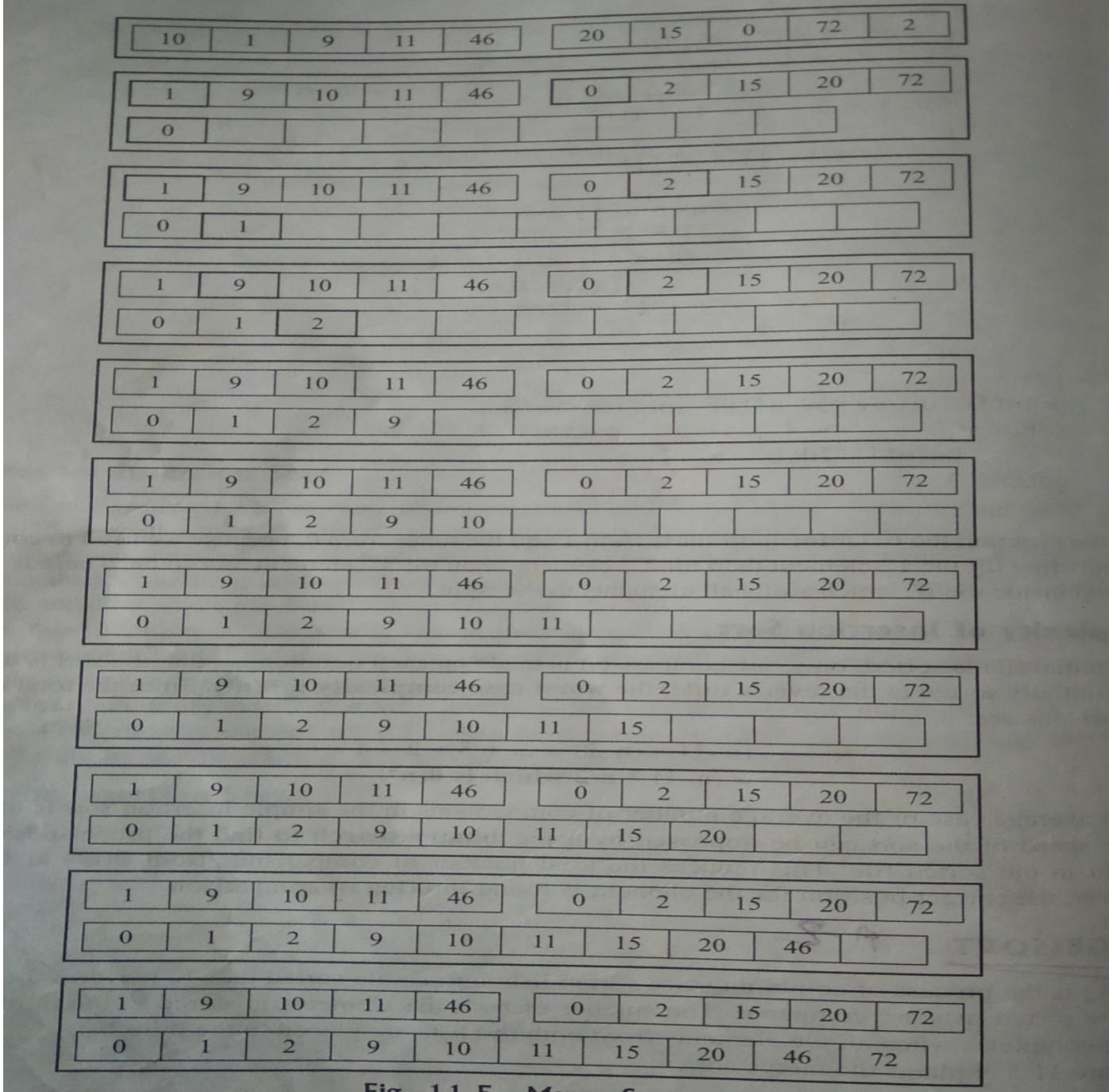


Fig. 11.5 *Merge Sort*

ALGORITHM FOR MERGE SORT

Algorithm for Merge Sort

1. The arrays x and y are sorted using any of the discussed algorithms.
2. The 0th element from the first array, 1, is compared with 0th element of second array 0. Since 0 is smaller than 1, 0 is moved in the third array.
3. Now the 0th element from the first array, 1, is compared with 1st element from the second array, 2. Since 1 is smaller than 2, 1 is placed in the third array.
4. Now the 1st element from the first array 9, is compared with the 1st element from the second array, 2. Since 2 is smaller than 9, 2 is placed in the third array.
5. Now the 1st element from the first array, 9, is compared with the 2nd element in the second array, 15. Since 9 is smaller than 15, 9 is placed in the third array.
6. The same procedure is repeated till the end of one of the arrays is reached. Now the remaining elements from the other array are placed directly into the third list as are already in sorted order.

RADIX SORT

RADIX SORT

The radix sort is used generally when we intend to sort a large list of names alphabetically. Radix in this case can be 26, as there are 26 alphabets. Therefore, if we want to arrange a list of names we can classify the names into 26 classes, where the first class consists of those names starting with '**A**'. The second class consists of names starting with '**B**' and so on.

All this can be done in first iteration. In the second iteration each class is alphabetized according to the second letter of the name, and so on. The radix sort method is used by a card sorter—A card sorter contains 13 receiving pockets labelled as follows:

9, 8, 7, 6, 5, 4, 3, 2, 1, 0, 11, 12, R (Reject)

The radix sort is based on the values of the actual digits in the positional representation. For example, the number 235 in decimal notation is written with a 2 in a hundred's position, 3 in ten's position and 5 in the unit's position.

The sorter used above uses decimal numbers where the radix is 10 and hence uses only the first 10 pockets of the sorter. Suppose we want to sort the following set of three digits numbers:

242, 986, 504, 428, 321

Pockets

Inputs	0	1	2	3	4	5	6	7	8	9
242			242							
986							986			
504					504					
428									428	
321		321								

First iteration

Pockets

Inputs	0	1	2	3	4	5	6	7	8	9
428			428							
986									986	
504	504									
242					242					
321			321							

Second iteration

Pockets

Inputs	0	1	2	3	4	5	6	7	8	9
986										986
242			242							
428					428					
321				321						
504						504				

Third iteration

Fig. 11.7 *Radix Sort*

The sorted list is as follows

986	504	428	321	242

ALGORITHM FOR RADIX SORT

Algorithm for Radix Sort

1. In the first iteration the elements are picked up and kept in various pockets checking their unit's digit.
2. The cards are collected from pocket 9 to pocket 0 and again they are given as input to sorter.
3. In second iteration, the ten's digits are sorted.
4. Repeat through step 2.
5. In the final iteration (in the above example) the digits at hundred's position are sorted.
6. Repeat through step 2.

SHELL SORT

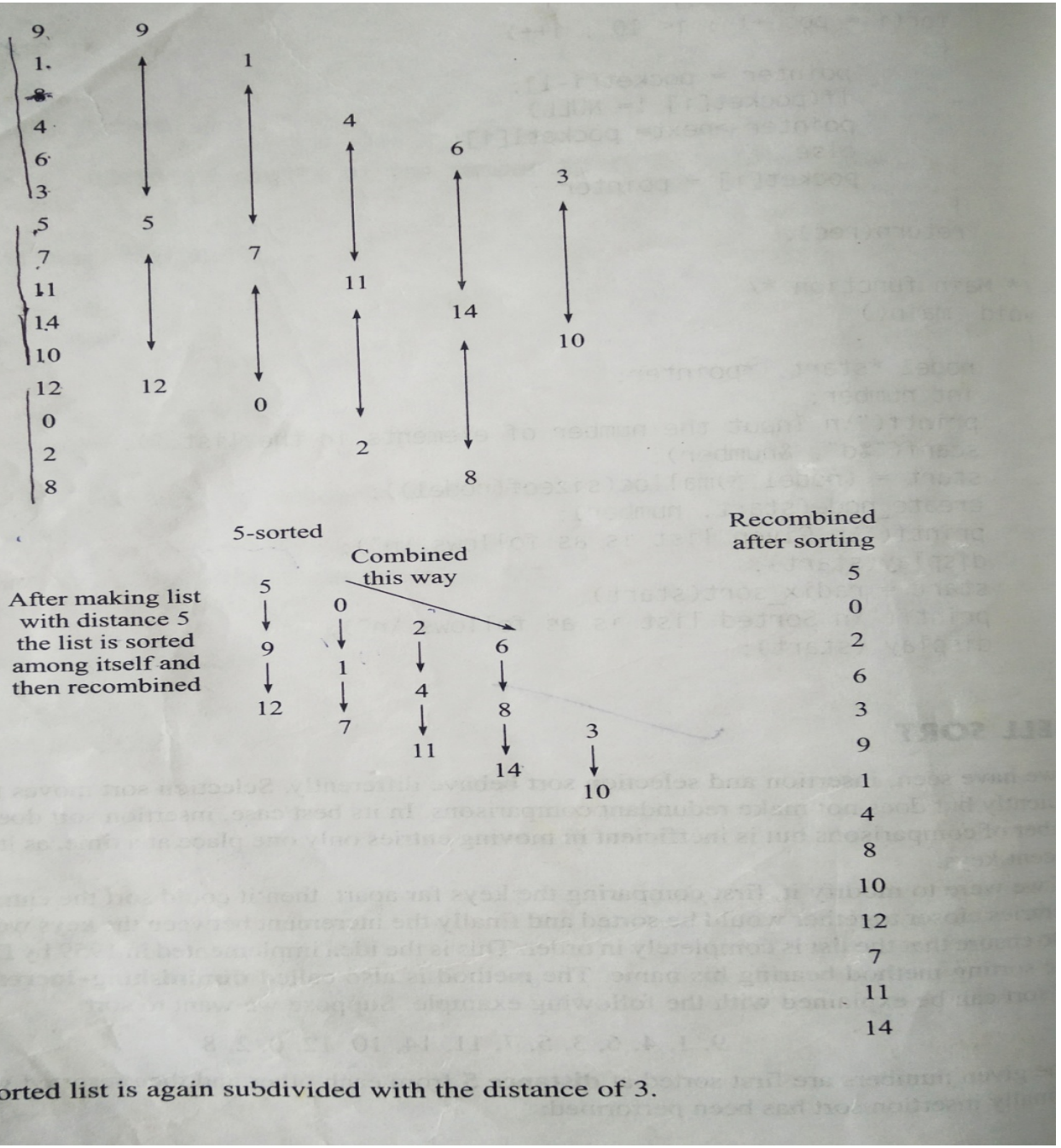
SHELL SORT

As we have seen, insertion and selection sort behave differently. Selection sort moves the entries very efficiently but does not make redundant comparisons. In its best case, insertion sort does the minimum number of comparisons but is inefficient in moving entries only one place at a time, as it compares only adjacent keys.

If we were to modify it, first comparing the keys far apart, then it could sort the entries. Afterwards the entries closer together would be sorted and finally the increment between the keys would be reduced to 1 to ensure that the list is completely in order. This is the idea implemented in 1959 by Donald. L. Shell in the sorting method bearing his name. The method is also called **diminishing–increment sort**. The shell sort can be explained with the following example. Suppose we want to sort:

9, 1, 4, 6, 3, 5, 7, 11, 14, 10, 12, 0, 2, 8

The given numbers are first sorted at **distance 5** from each other and then resorted with **distance 3** and finally insertion sort has been performed:



The above sorted list is again subdivided with the distance of 3.

ALGORITHM FOR SHELL SORT

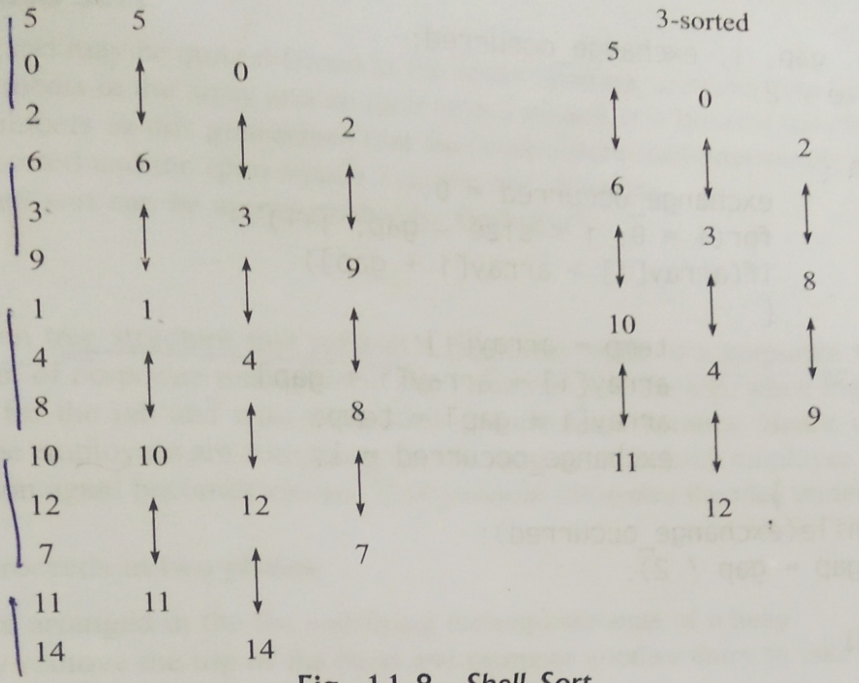


Fig. 11.8 *Shell Sort*

After getting the sorted list with distance 3 from each, other simple **insertion sort** is performed to get the final sorted list.

It is not mandatory to make 5, 3 and 1 as increments. Many other choices can also be made, but it should be considered that the choices like powers of 2 such as 8, 4, 2, 1 are not fruitful as the same keys compared in one pass would be compared again at the next pass. Therefore, making other choices may give better chance of obtaining new information from more of the comparisons.

The final iteration of shell sort has increment 1, shell sort really is insertion sort optimized by the preprocessing stage of first sorting sublists using larger increments. Although the preprocessing stage will speed up the sorting considerably by eliminating many moves of entries by only one position.

Algorithm for Shell Sort

1. In the first iteration, the elements are splitted with a gap of say, 5, 3, etc. and are sublisted.
2. The sublist obtained after splitting is again sorted.
3. The sorted sublist is recombined.
4. The sorted list, in the second iteration, is again sublisted with a gap (here, it is 3).
5. Repeat through steps 2 and 3.

HEAP SORT

HEAP SORT

Heap sort is based on tree structure that reflects a particular order of a corporate hierarchy. As in an organization structure of corporate management the President is at the top, when President retires Vice-President competes for the job and wins promotion and creates a vacancy. Hence vacancy continually appears at the top, the employees are competing for the promotion, as each employee reaches the "top of the heap" that position again becomes vacant. This example illustrates the idea underlying the heap sort method.

Heap sort thus proceeds in two phases:

- The entries are arranged in the list satisfying the requirements of a heap.
- We repeatedly remove the top of the heap and promote another entry to take its place.

In the second phase, we recall that the root of the tree (which is the first entry of the list as well as top of the heap) has the largest key. This key belongs to the end of the list.

The Heap sort can be understood from the following example—Assume that the array **x** that is to be sorted contains the following elements:

13, 4, 11, 15, 59, 27, 19, 3, 92, 5

The first step now is to create a heap from the array elements. For doing this imagine a binary tree that can be built from the array elements. The zeroth element would be the root element and left and right child of any element x[i] would be at **x[2*i+1]** and **x[2*i+2]** respectively.

The binary tree of the given array will be:

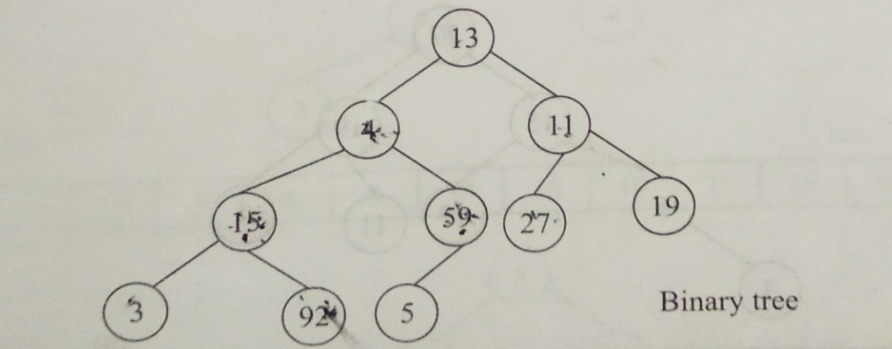


Fig. 11.9 *Binary Tree*

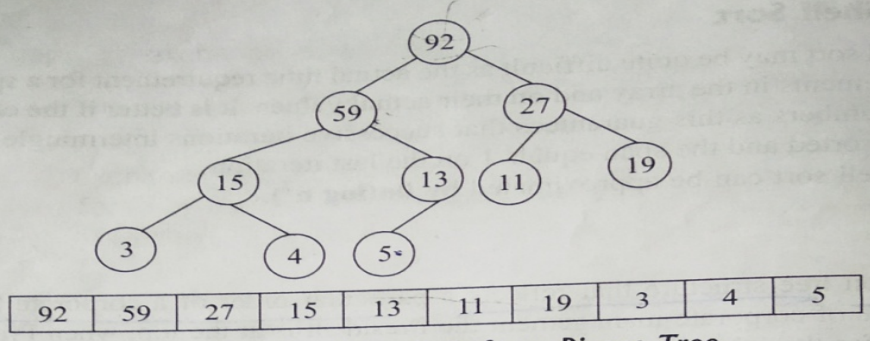
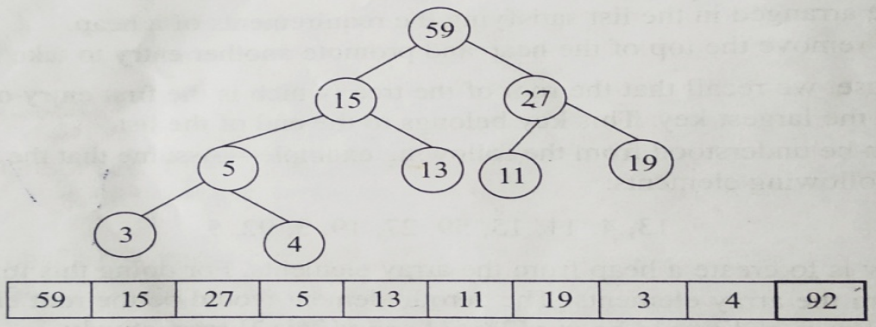
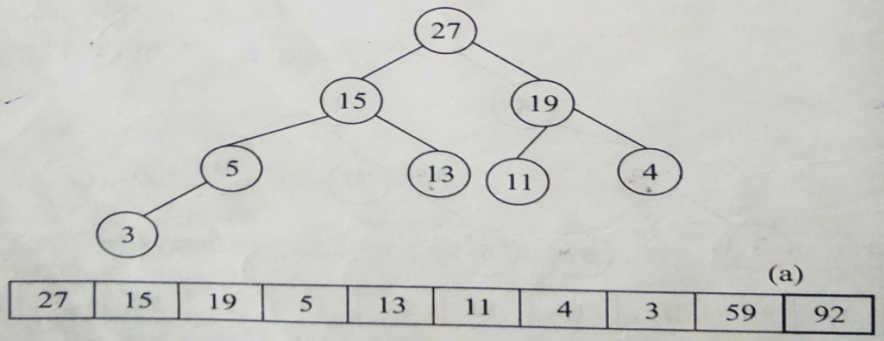


Fig. 11.10 *Heap Built from Binary Tree*

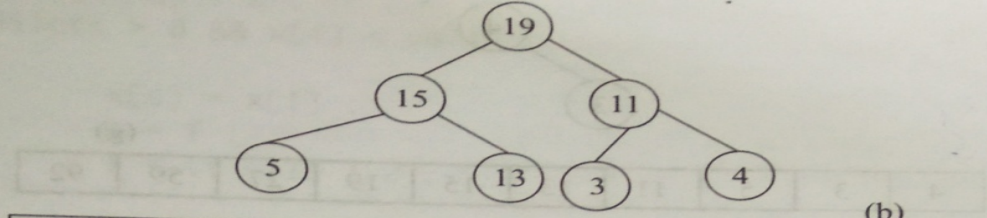
Now the root element 92 is moved to the last location by exchanging it with 5. Finally, 92 is eliminated from the heap by reducing the maximum index value of array by 3. The left elements are rearranged into heap.



Similarly, one by one root element of the tree is removed:

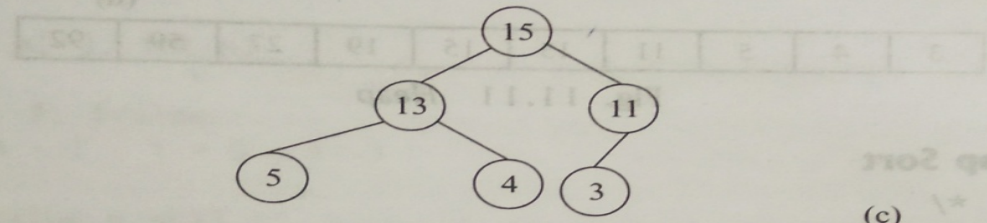


Heap after eliminating 59



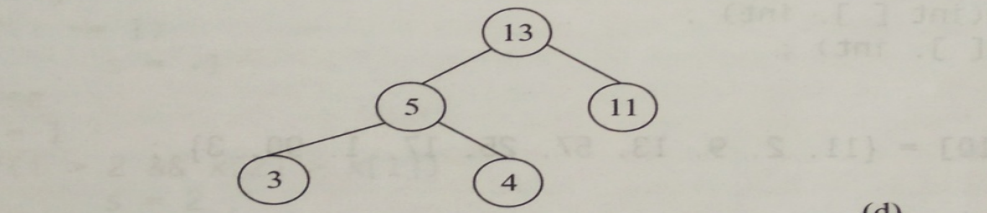
(b)

19	15	11	5	13	3	4	27	59	92



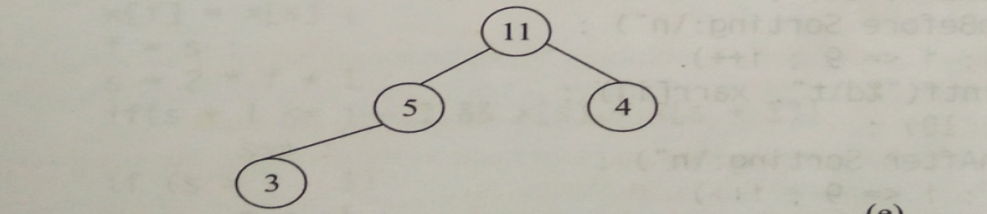
(c)

15	13	11	5	4	3	19	27	59	92



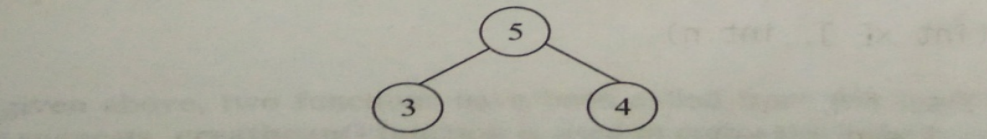
(d)

13	5	11	3	4	15	19	27	59	92



(e)

11	5	4	3	13	15	19	27	59	92



(f)

5	3	4	11	13	15	19	27	59	92

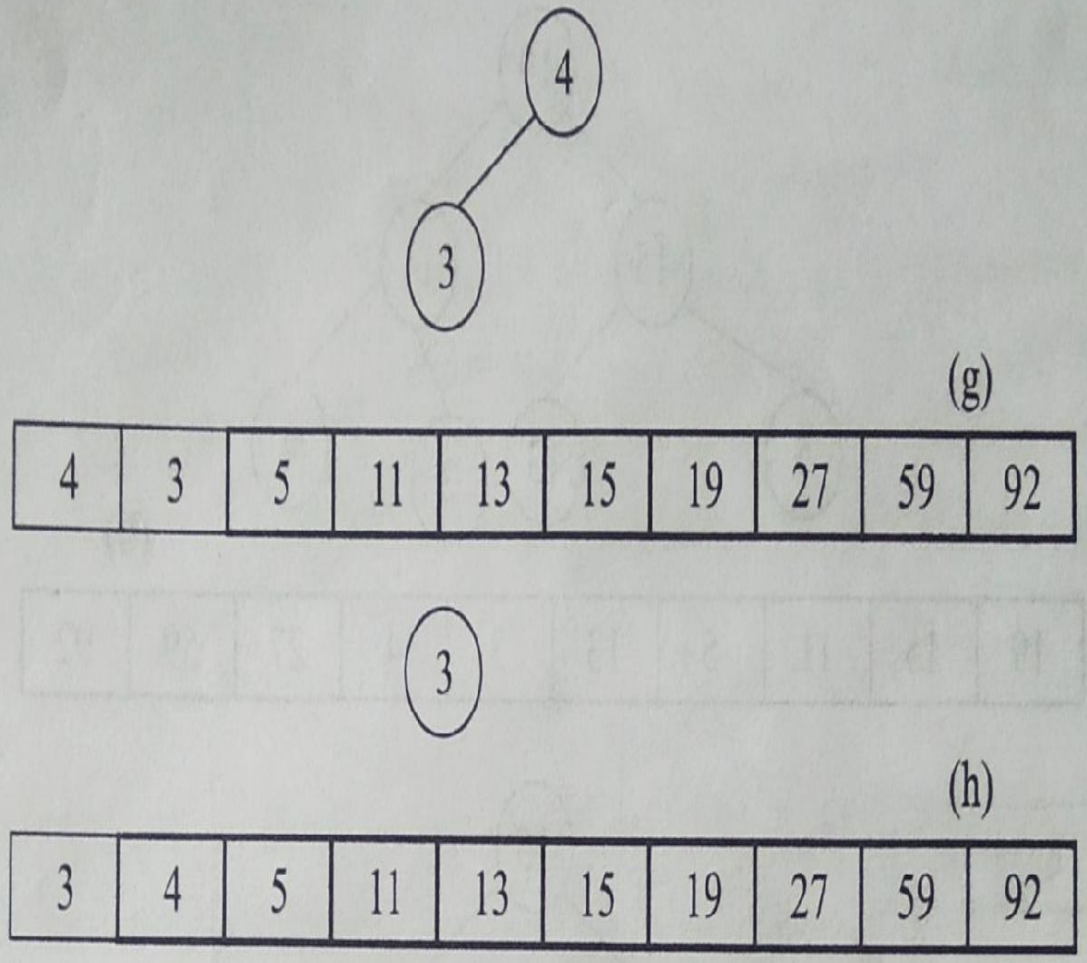


Fig. 11.11 *Heap*

THANK YOU